DashyDashboard

Deployment Guide

ASP.NET 7 + React on IIS / SQL Server

Broadridge Financial Solutions

June 2026

Document type:	Deployment / Operations Guide
Audience:	Windows Server Administrators
Classification:	Internal

Contents

Chapter 1

Prerequisites

Before beginning the deployment, ensure the following are in place on the target server.

1.1 Operating System

- Windows Server 2019 or Windows Server 2022 (Datacenter or Standard edition)
- Windows 10 or Windows 11 Pro is also supported for development/test deployments

1.2 SQL Server

- Any edition of Microsoft SQL Server must be installed and running (SQL Server 2019 or later recommended; SQL Server Express is acceptable)
- SQL Server Management Studio (SSMS) must be available to run the database setup script
- You must be able to connect to SQL Server with `Windows Authentication` from the target machine

1.3 Account Permissions

- You must be logged in as, or be able to run programs as, a **Local Administrator**
- The account running SSMS must have **sysadmin** rights on the SQL Server instance (or at least the ability to create databases and logins)

1.4 Deployment Package

The deployment package (the folder containing this document) must include:

- `app\` — the published ASP.NET application files (including `wwwroot\` with the React front-end)
- `setup.sql` — the database creation script
- This document

Note

No internet connectivity is required on the target server after the .NET 7 Hosting Bundle has been installed. All application binaries are self-contained in the `app\` folder.

Chapter 2

Install IIS (Web Server Role)

Internet Information Services (IIS) is the web server that will host the application. If IIS is already installed on this server, skip to Chapter 3.

1. Open **Server Manager**. It usually opens automatically after login; if not, click the Windows Start button and type *Server Manager*.
2. In the Server Manager dashboard, click **Manage** (top-right menu) and then select **Add Roles and Features**.
3. The *Add Roles and Features Wizard* opens. On the *Before You Begin* page, click **Next**.
4. On the *Installation Type* page, select **Role-based or Feature-based installation** and click **Next**.
5. On the *Server Selection* page, your local server should already be highlighted. Click **Next**.
6. On the *Server Roles* page, locate **Web Server (IIS)** in the list and tick its checkbox.
   - A dialog box will appear asking to add required features. Click **Add Features**.
   - Click **Next**.
7. On the *Features* page, leave all defaults as-is and click **Next**.
8. On the *Web Server Role (IIS)* information page, click **Next**.
9. On the *Role Services* page, expand **Security** and ensure the following are ticked:
   - **Windows Authentication**
   - Request Filtering (ticked by default)

   Click **Next**.
10. On the *Confirmation* page, click **Install**.
11. Wait for the installation progress bar to complete. When it reads *Installation succeeded*, click **Close**.

> **Note**
>
> After installation, you can verify IIS is running by opening a browser on the server and navigating to `http://localhost`. You should see the default IIS welcome page (blue IIS logo).

Chapter 3

Install the .NET 7 Hosting Bundle

The .NET 7 Hosting Bundle installs the ASP.NET Core runtime and the IIS integration module (AspNetCoreModuleV2) that allows IIS to host .NET applications.

3.1 Download

1. On a machine with internet access, open a browser and search for: *dotnet 7 hosting bundle download*
2. Navigate to the official Microsoft .NET download page (microsoft.com/net/download or dotnet.microsoft.com).
3. Under the **.NET 7** section, locate the **Hosting Bundle** for Windows and download the installer (`dotnet-hosting-7.x.x-win.exe`).
4. Transfer the installer file to the target server if you downloaded it on a different machine.

3.2 Install

1. Right-click the installer file and select **Run as administrator**.
2. Accept the license agreement and click **Install**.
3. Leave all options at their defaults and allow the installer to complete.
4. Click **Close** when the installation finishes.

3.3 Register with IIS

Important

If IIS was installed *before* the Hosting Bundle, you must restart IIS to register the ASP.NET Core Module. Skip this step if you installed the Hosting Bundle before IIS.

1. Click Start, type **IIS Manager**, and open *Internet Information Services (IIS) Manager*.
2. In the left pane, click on the server node (the topmost item with a computer icon).
3. In the centre pane, double-click **Modules**.
4. Scroll through the list and confirm an entry named **AspNetCoreModuleV2** exists.
5. If it is *not* listed:

a. Click Start, type **Command Prompt**, right-click it and choose **Run as administrator**.

b. Type `iisreset` and press Enter.

c. Wait for the confirmation message (*Internet services successfully restarted*).

d. Return to IIS Manager, refresh the Modules list, and confirm **AspNetCoreModuleV2** now appears.

Chapter 4

Create the Database

1. Open **SQL Server Management Studio (SSMS)** from the Start menu.
2. In the *Connect to Server* dialog:
   - Server type: **Database Engine**
   - Server name: enter the name of your SQL Server instance (e.g., `(local)` or `.\SQLEXPRESS` for Express)
   - Authentication: **Windows Authentication**
   - Click **Connect**
3. In the *Object Explorer* panel (left side), click **File** in the menu bar and select **Open** → **File...** (or press `Ctrl+O`).
4. Browse to the deployment package folder and select **setup.sql**. Click **Open**.
5. The SQL script will open in a query window. Click **Execute** in the toolbar (or press **F5**).
6. Wait for the script to finish. The *Messages* tab at the bottom should show output ending with lines similar to:

   ```
   (N row(s) affected)
   Command(s) completed successfully.
   ```

7. In *Object Explorer*, right-click **Databases** and select **Refresh**. Confirm **ClientsAppAttestation** now appears in the list.

Note

If the script reports that the database already exists, it is safe to re-run as long as the script uses `IF NOT EXISTS` guards. If you need a clean install, drop the existing database first via right-click → Delete in Object Explorer.

Chapter 5

Grant SQL Access to the IIS App Pool

The application runs under the IIS application pool identity (`IIS APPPOOL\DashyDashboardPool`). This Windows account must have permission to read and write the database.

Important

You must complete Chapter 7 (creating the application pool named `DashyDashboardPool`) before this chapter, because the login name is derived from the pool name. If you prefer, create the application pool first (Chapter 7, steps 1–4) and then return here.

1. In SSMS *Object Explorer*, expand your server node, then expand **Security**.
2. Right-click **Logins** and select **New Login...**
3. On the *General* page:
   - In the *Login name* field, type exactly: `IIS APPPOOL\DashyDashboardPool`
   - **Do not** use the Browse button — type the name manually
   - Leave *Windows authentication* selected
4. On the left panel, click **User Mapping**.
5. In the upper list of databases, tick the checkbox next to **ClientsAppAttestation**.
6. In the lower panel (*Database role membership for: ClientsAppAttestation*), tick the following roles:
   - **db_datareader**
   - **db_datawriter**
   - **db_ddladmin**
7. Click **OK** to create the login and user mapping.

Note

`db_ddladmin` is required on first launch because Entity Framework will apply database migrations automatically if the schema is not yet present or is out of date.

Chapter 6

Deploy the Application Files

1. Open **Windows Explorer**.
2. Navigate to `C:\inetpub\wwwroot` (this folder is created automatically by IIS).
3. Right-click in an empty area and select **New → Folder**. Name it **DashyDashboard**.
   - You may use a different path if your organisation's convention requires it (e.g., `D:\Apps\DashyDashboard`). Note the full path for use in Chapter 7.
4. Open the deployment package's `app\` folder.
5. Select all contents of the `app\` folder (**Ctrl+A**) and copy them (**Ctrl+C**).
6. Navigate back to `C:\inetpub\wwwroot\DashyDashboard` and paste (**Ctrl+V**).
7. (Optional but recommended) Inside the `DashyDashboard` folder, create a subfolder named **logs**. This is where IIS will write stdout logs if you ever enable them for troubleshooting.

After this step, the folder structure should look like:

```
C:\inetpub\wwwroot\DashyDashboard\
DashyDashboard.Api.dll
web.config
appsettings.Production.json
wwwroot\index.html
wwwroot\assets\...
logs\
```

Chapter 7

Create the IIS Site and Application Pool

7.1 Create the Application Pool

1. Open **IIS Manager** (Start → search *IIS Manager*).
2. In the left *Connections* pane, expand your server name.
3. Right-click **Application Pools** and select **Add Application Pool...**
4. In the dialog:
   - **Name:** `DashyDashboardPool`
   - **.NET CLR version: No Managed Code**
   - **Managed pipeline mode: Integrated**

   Click **OK**.

> **Note**
>
> Select *No Managed Code* because ASP.NET Core runs its own managed runtime. Selecting a .NET CLR version here would have no effect and could cause warnings.

7.2 Create the Website

1. In the left pane, right-click **Sites** and select **Add Website...**
2. Fill in the fields:
   - **Site name:** `DashyDashboard`
   - **Application pool:** click **Select...** and choose `DashyDashboardPool`
   - **Physical path:** click the **...** button and browse to the folder you created in Chapter 6 (e.g., `C:\inetpub\wwwroot\DashyDashboard`)
   - **Port:** `8080` (or another available port if 8080 is already in use)
   - Leave the IP address as **All Unassigned**

   Click **OK**.
3. IIS Manager will ask whether to stop the existing Default Web Site (if it is also using port 80). If you are using port 8080 you can leave the Default Web Site running.

Chapter 8

Enable Windows Authentication

1. In IIS Manager, click the **DashyDashboard** site in the left pane.
2. In the centre pane (Features View), locate the **Authentication** icon (under the *IIS* section) and double-click it.
3. In the *Authentication* list:
   - Click **Windows Authentication**. In the right *Actions* panel, click **Enable**.
   - Click **Anonymous Authentication**. In the right *Actions* panel, click **Disable**.
4. The final state should show:
   - Anonymous Authentication: **Disabled**
   - Windows Authentication: **Enabled**

Important

If *Windows Authentication* does not appear in the list, the Windows Authentication role service was not installed in Chapter 2. Return to Server Manager, modify the Web Server (IIS) role, and add **Security → Windows Authentication**.

Chapter 9

Verify the Deployment

1. On the server, open a web browser (Edge, Chrome, or Firefox).
2. Navigate to: `http://localhost:8080`
3. Your browser may display a Windows credentials prompt. Enter your Windows username and password.
4. The **DashyDashboard** login page should appear in the browser.
5. The application uses Windows Authentication. Once the browser has authenticated you, it passes your identity to the application, which resolves your account from the `Users` table in the database. If your Windows account has a corresponding entry, you will be logged in automatically.

Note

When accessing the application from other machines on the network, use the server's hostname or IP address instead of `localhost`, for example: `http://servername:8080`. Ensure that port 8080 is open in the Windows Firewall (Windows Defender Firewall → Advanced Settings → Inbound Rules → New Rule → Port 8080 TCP).

Chapter 10

Troubleshooting

The table below lists the most common deployment problems and their solutions.

Symptom	Likely Cause	Fix
HTTP 500.21 — ANCM handler not found	IIS was installed *after* the Hosting Bundle	Open a Command Prompt as Administrator and run `iisreset`. This registers AspNetCoreModuleV2 with IIS.
HTTP 500.30 — App failed to start	Database connection failing, or a startup exception in the application	Verify SQL Server is running. Open `appsettings.Production.json` in the deployment folder and confirm the `Default` connection string points to the correct server. Enable stdout logging temporarily: open `web.config`, set `stdoutLogEnabled="true"`, restart the site, and check the `logs\` folder.
HTTP 400 Bad Request	App pool identity not granted access to SQL Server	Complete Chapter 5 in full. Ensure the login name is typed as `IIS APPPOOL\DashyDashboardPool` exactly.
HTTP 401 Unauthorized	Windows Authentication not enabled or Anonymous Authentication still enabled	Complete Chapter 8. Verify the Authentication feature settings match the required state (Windows Auth Enabled, Anonymous Disabled).

Symptom	Likely Cause	Fix
Page loads but shows only a blank white screen or "Loading…" indefinitely	React front-end files (`wwwroot\`) not copied, or the API cannot reach the database	Verify that `wwwroot\index.html` exists in the deployment folder (Chapter 6). If it exists, open browser developer tools (F12) and check the Network tab for failing API requests.
"Loading…" spinner never resolves after login	Application pool stopped or crashed	In IIS Manager, right-click **DashyDashboardPool** under Application Pools and select **Start**. If it immediately stops again, enable stdout logging and inspect the error.
Windows credentials prompt loops endlessly	Kerberos/NTLM negotiation issue (common when accessing via IP address rather than hostname)	Access the application using the server's hostname instead of its IP address. Alternatively, add the site to Internet Explorer's *Local Intranet* zone via Internet Options.
Database exists but tables are missing	`setup.sql` was not fully executed	Re-open `setup.sql` in SSMS, select all text (`Ctrl+A`), and click Execute (F5). Check for any error messages in the *Messages* tab.

10.1 Useful Locations

- **Application files:** `C:\inetpub\wwwroot\DashyDashboard\`
- **IIS Manager:** Start → search *IIS Manager*
- **Windows Event Log:** Start → search *Event Viewer* → Windows Logs → Application (ASP.NET Core errors are logged here)
- **IIS logs:** `C:\inetpub\logs\LogFiles\W3SVC{n}\`
- **Connection string:** `app\appsettings.Production.json`

Part I

Application Reference

Chapter 11

Application Overview

DashyDashboard is a tool attestation management system for Broadridge. It records, tracks, and reports whether associates actively used the client-facing tools to which they had access during a given period.

11.1 Purpose

Each calendar month, an administrator opens a new *attestation cycle*. During that cycle, every associate whose account appears in the system receives a personalised list of the client tools they were granted access to. The associate reviews each tool and confirms:

1. Whether they actually had access to the tool during the cycle (the system pre-fills this based on the access records, but the associate may dispute it).
2. Whether they used that tool during the cycle.

Each attestation row can also carry a free-text remark. Once an associate has reviewed all their tools they submit their attestation; the submission timestamp is recorded and the associate's status updates to *Submitted*.

11.2 Role Summary

- **Associates** — log in, attest their own tools.
- **Managers** — see a progress dashboard for their direct reports and are alerted if any report disputes their access.
- **Administrators (Admin)** — full department-wide completion dashboard; can add new tools.
- **GFHs and GFH-Delegates** — department-level dashboard, same scope as Admin.
- **IFHs** — department-scoped reporting, configurable access level.

11.3 Authentication

The application relies entirely on **Windows Authentication** via IIS. No username or password form is presented to the user. IIS authenticates the browser session using Kerberos/NTLM and passes the Windows identity to the application, which resolves the corresponding account from the `dbo.Users` table by matching the `WindowsID` column. Every login attempt (successful or failed) is recorded in `dbo.LoginLogs`.

Chapter 12

Database Schema

The application database is named **ClientsAppAttestation**. It contains the ten tables described below. All string primary keys that may carry leading zeros (such as client identifiers) are stored as `VARCHAR` to preserve those characters exactly.

12.1 dbo.Departments

Holds the list of business departments. Every user, client, tool, and access record is linked to a department, enabling scoped reporting for managers and GFHs.

Column	Notes
`DepartmentID`	`INT IDENTITY` — primary key, auto-incremented.
`DepartmentName`	`VARCHAR(150) NOT NULL` — human-readable name of the department.

12.2 dbo.Users

Stores one row per associate. `AssociateID` is the business primary key and is treated as a string throughout the system so that alphanumeric and zero-padded identifiers are preserved without corruption.

Column	Notes
`AssociateID`	`VARCHAR(50) NOT NULL` — primary key (not an identity column).
`FirstName`	`VARCHAR(100)`
`LastName`	`VARCHAR(100)`
`EMailAddr`	`VARCHAR(255)` — corporate email address.
`WindowsID`	`VARCHAR(200)` — Windows domain account used to match the IIS authenticated identity at login.

Column	Notes
ManagerID	VARCHAR(50) NULL — self-referencing foreign key to dbo.Users.AssociateID. Any user whose AssociateID appears as another user's ManagerID is automatically treated as a manager by the application.
DepartmentID	INT — foreign key to dbo.Departments.
IsActive	BIT NOT NULL DEFAULT 1 — set to 0 to deactivate an account without deleting it.

12.3 dbo.Clients

Holds the client entities whose tools associates must attest. ClientID is stored as VARCHAR so that identifiers containing leading zeros or non-numeric characters (for example BARCLAYS or DTC-US) are preserved as-is.

Column	Notes
ClientID	VARCHAR(50) NOT NULL — primary key (not an identity column).
ClientName	VARCHAR(255) NOT NULL
DepartmentID	INT — foreign key to dbo.Departments.
IsActive	BIT NOT NULL DEFAULT 1

12.4 dbo.ClientTools

Defines the set of tools that belong to each client. A single tool is uniquely identified by the combination of its client and a numeric tool identifier.

Column	Notes
ClientID	VARCHAR(50) NOT NULL — part of composite primary key; foreign key to dbo.Clients.
ToolID	INT NOT NULL — part of composite primary key.
ToolName	NVARCHAR(200) NOT NULL — display name of the tool.
DepartmentID	INT — foreign key to dbo.Departments.

Primary key: composite (ClientID, ToolID).

12.5 dbo.UsersToolAccess

Records which associate has access to which client tool, when that access was granted, and whether it is a primary or secondary access tier. One row represents one associate-client-tool access grant.

Column	Notes
`AssociateID`	`VARCHAR(50) NOT NULL` — part of composite primary key; foreign key to `dbo.Users`.
`ClientID`	`VARCHAR(50) NOT NULL` — part of composite primary key; foreign key to `dbo.Clients`.
`ToolID`	`INT NOT NULL` — part of composite primary key.
`Tier`	`NVARCHAR(50) NOT NULL` — `Primary` or `Secondary`.
`GivenDate`	`DATE NOT NULL` — date access was granted.
`AccessTo`	`DATE NULL` — date access is scheduled to expire; `NULL` means open-ended.
`GrantedBy`	`VARCHAR(50) NULL` — foreign key to `dbo.Users.AssociateID`; the associate who granted this access.
`DepartmentID`	`INT` — foreign key to `dbo.Departments`.

Primary key: composite `(AssociateID, ClientID, ToolID)`.

12.6 dbo.Cycles

Defines each attestation cycle. The application reads the most recent active cycle automatically; no application restart is required when a new cycle is added.

Column	Notes
`CycleID`	`INT IDENTITY` — primary key, auto-incremented.
`CycleName`	`NVARCHAR(100) NOT NULL` — descriptive label, e.g. "Q2 2026 Attestation".
`StartDate`	`DATE NOT NULL` — first day of the attestation window.
`EndDate`	`DATE NOT NULL` — last day of the attestation window.
`DueDate`	`DATE NOT NULL` — deadline by which all associates must have submitted.

12.7 dbo.ToolCycleAttestation

The central fact table. One row exists per associate-tool combination within a cycle. Rows are created by the application when a cycle opens and are updated as associates attest each tool.

Column	Notes
`CycleID`	`INT NOT NULL` — part of composite primary key; foreign key to `dbo.Cycles`.
`AssociateID`	`VARCHAR(50) NOT NULL` — part of composite primary key; foreign key to `dbo.Users`.
`ClientID`	`VARCHAR(50) NOT NULL` — part of composite primary key; foreign key to `dbo.Clients`.
`ToolID`	`INT NOT NULL` — part of composite primary key.
`AttestationStatus`	`NVARCHAR(50) NOT NULL DEFAULT 'Pending'` — transitions to `Submitted` when the associate submits.
`UsedThisCycle`	`BIT NULL` — `1` = used, `0` = not used, `NULL` = not yet answered.
`HadAccess`	`BIT NOT NULL DEFAULT 1` — system-populated from `dbo.UsersToolAccess`; the associate may set this to `0` to dispute that they had access.
`Remarks`	`NVARCHAR(500) NULL` — free-text comment from the associate.
`SubmittedAt`	`DATETIME NULL` — populated when the associate submits.

Primary key: composite `(CycleID, AssociateID, ClientID, ToolID)`.

12.8 dbo.AttestationLogs

Provides a summary-level audit trail of each associate's submission event. One row is written per associate per cycle at the moment of submission.

Column	Notes
`LogID`	`INT IDENTITY` — primary key, auto-incremented.
`CycleID`	`INT NOT NULL` — foreign key to `dbo.Cycles`.
`AssociateID`	`VARCHAR(50) NOT NULL` — foreign key to `dbo.Users`.
`SubmittedAt`	`DATETIME NOT NULL`

Column	Notes
ToolCount	INT NOT NULL — number of tools included in the submission.
Summary	NVARCHAR(100) NULL — short human-readable summary generated at submission time.

12.9 dbo.SuperUsers

Grants elevated roles to specific associates. An associate may hold more than one role (for example, both GFH and IFH) by having multiple rows, one per role. The unique index on (AssociateID, RoleName, DepartmentID) prevents duplicate grants.

Column	Notes
SuperUserID	INT IDENTITY — primary key, auto-incremented.
AssociateID	VARCHAR(50) NOT NULL — foreign key to dbo.Users.
RoleName	VARCHAR(50) NOT NULL — one of Admin, GFH, GFHDelegate, IFH.
DepartmentID	INT NOT NULL — foreign key to dbo.Departments; scopes the role to a department. Admin role ignores this constraint and sees all departments.
AccessLevel	VARCHAR(50) NOT NULL — Full, ReadOnly, or Approver.
IsActive	BIT NOT NULL DEFAULT 1 — set to 0 to revoke the role without deleting the row.

Unique index: (AssociateID, RoleName, DepartmentID).

12.10 dbo.LoginLogs

Records every login attempt. AssociateID is nullable so that failed attempts (where the Windows identity did not match any user in dbo.Users) are still captured with a Status of UserNotFound.

Column	Notes
LoginLogID	INT IDENTITY — primary key, auto-incremented.
AssociateID	VARCHAR(50) NULL — foreign key to dbo.Users; NULL when no matching user was found.
LoginTime	DATETIME NOT NULL — timestamp of the attempt.

Column	Notes
`Status`	`NVARCHAR(50) NOT NULL` — `Success` or `UserNotFound`.

Chapter 13

User Roles

Access to application features is determined by the user's role. The base role (Associate) is implicit for every account in `dbo.Users`. Elevated roles are stored in `dbo.SuperUsers` with the exception of Manager, which is derived automatically from the `ManagerID` hierarchy in `dbo.Users`.

13.1 Associate

The default role. Any account present in `dbo.Users` with `IsActive = 1` is an Associate.

- Logs in via Windows Authentication. The application resolves identity by matching the IIS-supplied Windows account name against `dbo.Users.WindowsID`.
- Can view and submit attestations for their own tool access in the current cycle.
- Attestation is a two-step process per tool:
  1. **Had Access?** — the system pre-populates this as true based on `dbo.UsersToolAccess`. The associate may set it to false to dispute the access record.
  2. **Used This Cycle?** — this field is disabled if "Had Access" is false.
- The Remarks field is always editable regardless of the other field values.

13.2 Manager

No `SuperUsers` row is required. An associate becomes a Manager automatically when their `AssociateID` appears as the `ManagerID` of at least one other active user.

- Sees a team dashboard listing each direct report and their attestation completion status for the current cycle.
- Receives a mismatch notification if any direct report has disputed their access (`HadAccess = 0`); the manager can drill in to view the dispute details.
- A Manager is also an Associate and can attest their own tools in the same session.

13.3 Admin

Stored in `dbo.SuperUsers` with `RoleName = 'Admin'`.

- Full department-wide overview dashboard: completion rates and per-client breakdowns across all departments.
- Can add new tools via the **Add Tool** button. The form requires selecting a **Client**, a **Department**, and entering a **Tool Name**. Admins may add tools to any client

in any department.

- The `DepartmentID` column on the Admin's `SuperUsers` row is recorded but does not restrict visibility — Admins see all departments.

13.4 GFH (Global Functional Head)

Stored in `dbo.SuperUsers` with `RoleName = 'GFH'`.

- One GFH is displayed per department on the Admin dashboard.
- Has the same dashboard access as Admin: completion rates and per-client breakdowns.
- Can add new tools via the **Add Tool** button, scoped to their own department.
- Intended for the senior leader who is accountable for a department's attestation completion.

13.5 GFH-Delegate

Stored in `dbo.SuperUsers` with `RoleName = 'GFHDelegate'`.

- Carries identical system access to GFH — same dashboard view, same department scope, and the same ability to add tools via the **Add Tool** button (scoped to their department).
- Used when a GFH delegates day-to-day attestation oversight to a colleague. The delegate acts on behalf of the GFH but is not themselves the GFH.
- Multiple GFH-Delegates may be assigned to the same department by inserting one `dbo.SuperUsers` row per delegate.
- To assign a GFH-Delegate: insert a row into `dbo.SuperUsers` with `RoleName = 'GFHDelegate'`, the delegate's `AssociateID`, and the appropriate `DepartmentID`.

13.6 IFH (Internal Functional Head)

Stored in `dbo.SuperUsers` with `RoleName = 'IFH'`.

- Currently reserved for department-scoped reporting.
- The exact access level is configurable via the `AccessLevel` field: `Full`, `ReadOnly`, or `Approver`.

Chapter 14

Adding a User to a Role

Role assignments are managed directly in SQL Server Management Studio. No application deployment or restart is required.

14.1 Prerequisites

- The associate must already have a row in `dbo.Users` with `IsActive = 1`.
- You must know the correct `DepartmentID` integer from `dbo.Departments`.
- You must be connected to the **ClientsAppAttestation** database in SSMS with sufficient permissions to edit table rows (`db_datawriter` or higher).

14.2 Steps

1. Open **SSMS** and connect to the database server using Windows Authentication.
2. In *Object Explorer*, expand **Databases** → **ClientsAppAttestation** → **Tables**.
3. Right-click **dbo.SuperUsers** and select **Edit Top 200 Rows**.
4. Scroll to the first empty row at the bottom of the grid.
5. Enter values for the following columns:
   - **AssociateID** — the associate's ID exactly as it appears in `dbo.Users`. Must not be null.
   - **RoleName** — one of: `Admin`, `GFH`, `GFHDelegate`, `IFH`.
   - **DepartmentID** — the integer department ID from `dbo.Departments`.
   - **AccessLevel** — `Full` for Admin/GFH/GFHDelegate; `Full`, `ReadOnly`, or `Approver` for IFH.
   - **IsActive** — `1`.
6. Press **Tab** or click another row to commit the new entry.

Note

The change takes effect immediately. The associate will see the elevated role on their next login without any application restart.

14.3 Removing a Role

Set `IsActive = 0` on the relevant row rather than deleting it. This deactivates the role while preserving the audit trail of when and to whom it was previously granted.

Chapter 15

Attestation Cycle Management

15.1 Opening a New Cycle

Cycles are managed directly in the database via SSMS. The application reads the most recently opened cycle automatically; no code change or restart is required.

1. Open **SSMS** and connect to the database server.
2. In *Object Explorer*, expand **Databases** → **ClientsAppAttestation** → **Tables**.
3. Right-click **dbo.Cycles** and select **Edit Top 200 Rows**.
4. Scroll to the first empty row and enter:
   - **CycleName** — a descriptive label, for example `Q2 2026 Attestation`.
   - **StartDate** — first day of the attestation window (format `YYYY-MM-DD`).
   - **EndDate** — last day of the attestation window.
   - **DueDate** — deadline by which all associates must submit.
5. Press **Tab** or click another row to commit.

Note

Associates see the new cycle and their updated tool list on their next login. No data from previous cycles is overwritten; each cycle's attestation records are fully independent.

15.2 Cycle Lifecycle

When a cycle is active, associates can:

- View the full list of client tools they have access to for that cycle.
- Set "Had Access" and "Used This Cycle" for each tool.
- Add or edit remarks at any time before the due date.
- Submit their attestation (sets `AttestationStatus = 'Submitted'` and records `SubmittedAt`).

A submission entry is also written to `dbo.AttestationLogs` recording the total number of tools attested and a brief summary. Managers and Admins can monitor completion progress in real time via their dashboards.